AI-Driven Job Market Intelligence: Predicting and Recommending Career Opportunities for Students and Graduates.

Mineh Issa Gholian

24018724

Github Repository:

https://github.com/Mineh-2005/Digital-Systems-Project

UFCFXK-30-3

Digital Systems Project

UWE Bristol | University of the West of England

Abstract

This report presents MarketMatch, an AI-driven job market intelligence system developed for the DSP-UFCFXK-30-3 module at UWE Bristol, which applies a weighted Natural Language Processing recommendation system to address information asymmetry in the UK graduate labour market and delivers personalized, explainable job matches ranked by a three-component score combining TF-IDF cosine similarity, skill overlap, and degree qualification fit. The system was built using Python 3 with Flask, scikit-learn, and Pandas, connecting to a curated 1,000-record UK job dataset spanning 35 industries and to live vacancy and salary data via the Adzuna Developer API, where functional and non-functional requirements were implemented and verified through multi-profile manual testing. Results demonstrate sub-200 millisecond recommendation response times, coherent score decomposition across diverse graduate profiles, and actionable market intelligence output, with critical limitations including the absence of persistent storage, a heuristic forecasting model, and the semantic constraints of TF-IDF relative to transformer-based alternatives identified and prioritised for future development.

Acknowledgements

I would like to express my sincere gratitude to all those who contributed to the successful completion of this project and the MarketMatch system.

First, I would like to thank my supervisor, Dr. Elias Pimenidis, for his invaluable support, guidance, and encouragement throughout this project, which contributed to its successful completion. The University of the West of England, (UWE Bristol) for providing the academic foundation, computational resources, and learning environment that made this project possible. I am grateful to the module team for Digital Systems Project for their co-operation, guidance, and continuous support throughout the academic year.

Thanks to Adzuna for providing access to the Adzuna Developer API (Adzuna, 2024), which supplied the live vacancy and salary data that powers the market intelligence and prediction features of MarketMatch.

I thank the broader academic community whose published research formed the theoretical backbone of this project, for their foundational work on information retrieval, and the many researchers cited in this report whose studies on algorithmic hiring, content-based recommender systems, and NLP preprocessing informed key design decisions.

Finally, I want to thank my friends for their willingness in sharing their ideas and honest feedback on the interface design, and their moral support during challenging times.

Table of Contents

Abstract 1
Acknowledgements ..... **Error! Bookmark not defined.**
Table of Contents 3
Table of Figures 5
Introduction 7
Literature Review 8
Job Application and Smart Recommender Systems. 8
Introduction to Algorithmic recruitment 8
2.2.1 Existing Solutions and Market Gaps 8
Natural Language Processing (NLP) Pipeline 9
2.3.1 Text Preprocessing and Normalization 9
Vectorization and Feature Extraction 9
2.4.1 Bag of Words (BoW) vs. TF-IDF 9
2.4.2 N-Grams 10
Similarity Algorithms and Ranking 10
2.5.1 Euclidean Distance vs. Cosine Similarity 10
2.5.2 Weighted Scoring Logic 10
Design 11
Research Design and Philosophical Stance 11
Data Collection and Curation 11
Text Preprocessing and Feature Engineering 12
Scoring Algorithm and Theoretical Justification 13
Real-Time Market Intelligence Methodology 13
Tools, Technologies and Libraries 14
Implementation and Testing 15
System Architecture Overview 15
NLP Pipeline Implementation 16
CV Processing and File Handling 17
Frontend Implementation and User Experience 18
Job Recommendation and Explainability Interface 19
Market Predictions and Administrative Features 20
Project Evaluation 22
Functional Requirements Verification 22

Match Score Accuracy Analysis 23
Performance and Responsiveness 24
Market Intelligence Results 24
Critical Evaluation of the NLP Approach 24
Further Work and Conclusion 26
Limitations 26
Ethical Considerations 26
Conclusion 26
Glossary 27
Table of Abbreviations 30
References / Bibliography 32
Appendices 36
Appendix A: First Appendix 36
Appendix B: System Screenshots 38
Appendix C: Algorithm Pseudocode 41
Appendix D: Dataset Sample 42

Table of Figures

Library / Tool	Language	Justification
Flask	Python	Lightweight framework sufficient for serving templates and REST endpoints without the overhead of Django
scikit-learn (TfidfVectorizer, cosine similarity)	Python	Industry-standard implementation of TF-IDF and cosine similarity; well-documented and performant on datasets of this scale
pandas	Python	Efficient tabular data handling for loading, filtering, and transforming the job dataset
pypdf / python-docx	Python	Extracts raw text from uploaded PDF and DOCX CV files respectively, supporting the three most common CV formats (FR02)
python-dotenv	Python	Loads environment variables (API keys, admin email) from a .env file, keeping credentials out of source code
Adzuna REST API	External	Provides live UK job market data including real-time job counts, salary ranges, and redirect URLs for job applications
Tailwind CSS (CDN)	CSS	Utility-first CSS framework used for rapid UI development; loaded from CDN to avoid a build step
localStorage (browser API)	JavaScript	Used for client-side state persistence across pages (user name, job results, saved jobs) without requiring a database

Table 1: Displaying list of libraries and tools used.

Endpoint	Method	Purpose
/api/signup	POST	Registers a new user and stores credentials in-memory (FR01)
/api/login	POST	Authenticates a user and creates a Flask session (FR01)
/api/recommend	POST	Accepts CV file and/or degree and skills input, runs the NLP pipeline, and returns ranked job results (FR02–FR06, FR09, FR11)
/api/profile (GET/POST)	GET/POST	Retrieves and updates the current user's session profile
/api/market	GET	Fetches live job counts, salary data, and a 6-month demand forecast from the Adzuna API (FR04, FR09)
/api/live-jobs	GET	Returns live job listings with redirect URLs for external applications
/api/admin/upload-dataset	POST	Allows an authenticated admin to replace the job dataset without restarting the server (FR07)
/api/dataset-version	GET	Returns the current dataset version metadata for the admin panel (NR09)
/api/forgot-password	POST	Accepts an email and returns a success response (stub endpoint for future email integration)

Table 2: List of api endpoints with each functionality.

Introduction

The graduate labour market in the United Kingdom has become a structurally volatile arena, one characterised by acute information asymmetry between employers and job seekers, rapidly shifting skill demands, and an overwhelming volume of vacancy listings that no individual can reasonably process unaided, and it is precisely this convergence of pressures that motivates the development of an intelligent, data-driven intermediary system. Research by Adzuna (2025) confirms that graduate vacancies declined by 44.8% year-on-year across 2025, illustrating that competition has intensified precisely at the moment when universities are producing record numbers of graduates, and this structural tension demands computational solutions that go far beyond a simple keyword-based job board. The present report documents the design, development, and critical evaluation of **MarketMatch**, an AI-powered job recommendation system that uses Natural Language Processing to match graduates against a curated UK job dataset and live market intelligence retrieved via the Adzuna Developer API (Adzuna, 2024).

Existing platforms such as LinkedIn, indeed, and Reed operate through broad keyword matching and employer-sponsored promotion algorithms, which means that candidates whose vocabulary differs from the exact phrasing in a job description are systematically disadvantaged regardless of their actual suitability, a limitation extensively critiqued in empirical literature on algorithmic hiring (Raghavan et al., 2020). MarketMatch addresses this by deploying a weighted, multi-signal scoring function that combines TF-IDF cosine text similarity, skill overlap, and a binary degree qualification check, thereby capturing semantic, and eligibility dimensions of fit simultaneously, and this triples the information density of any system. The Flesch Reading Ease scores of existing automated pipelines and their opacity toward users represent an additional problem that MarketMatch directly counters through an explainability panel on every job detail page, showing users the exact arithmetic of their match score.

The system comprises a Flask backend, NLP pipeline (NLP.py), a UK job dataset, and live market forecasting via the Adzuna API. The objectives were: personalised ranked job matches, actionable skills-gap feedback, and live labour market insights by qualification.

Literature Review

Job Application and Smart Recommender Systems.

The whole spectrum of the recruitment has fundamentally shifted over the last decade, especially post covid. The manual labour process on relying on newspaper, TV, journals, books, and other traditional approaches, has narrowed the broad options for the prospective candidates. The advent of the internet has given the immunity to recruiters and the job seekers, allowing for a global job market where any candidates could reach out instantly to the employers. (Barber,2016). But a new problem brought about by digitization is information overload. With thousands of positions being posted every day, it became ineffective for recruiters and candidates to manually search for relevant opportunities. (Barber,2016).

To address this, smart job systems did borrow the idea of utilising AI to automate the discovery process, predicting the matchmaking skills bases on profile data of the users, rather than search queries done manually. (Nadar et al., 2024). Smart job recommender systems proactively find appropriate opportunities by analysing user profile data, in contrast to traditional search-based systems that require users to submit job searches. This strategy may shorten the time needed to find a job and enhance the calibre of candidate-job matching. (Nadar et al., 2024)

Introduction to Algorithmic recruitment

Existing Solutions and Market Gaps

Most of the early platform recruitment relied on “Boolean” or matching systems keyword. These systems primarily function on by checking if a specific keyword (“python”) exists in both Job description and the CV. Well primarily, it’s simple to implement it, however these systems suffer from “Vocabulary mismatch problem”. (Bharath et al., n.d.). As noted by (Milvus.io, 2026) A keyword-based system will not match an applicant who characterizes themselves as a "Coder" even though the job description calls for a "Software Developer," even though the semantic meaning is the same.

The contemporary solutions with the likes of LinkedIn, deploys Deep Learning and transformer models like BERT the semantic gap. (Bevara et al., 2025). However, these models so called “Black Box” models does not give full overview of their “explainability”. (Tang et al., 2025). Candidates frequently receive job recommendations or rejections without knowing why, which erodes systemic confidence. (Tang et al., 2025). Furthermore, most commercial platforms use Collaborative Filtering, which makes job recommendations based on user activity in the past (e.g., "users who clicked this also clicked that"). This causes the "Cold Start Problem," a major obstacle for graduates and students (James, 2025). Since the idea is not elaborative, coupled with students and graduates' lack of professional history, or platform interaction data, most of these collaborative algorithms fail to generate accurate recommendations for them (James, 2025).

Consequently, there is a disparity for content - based filtering system catered for students and graduates. Such a system bypasses the cold start problem by scanning and matching the content of a CV and a degree befitting to job descriptions, without needing historical behavioural data. (James 2025)

Natural Language Processing (NLP) Pipeline

Text Preprocessing and Normalization

Converting content-based, including resumes and job descriptions, into machine-processable structured representations is necessary for developing a content-based recruitment system. This means that before semantic embedding and similarity computation, there must be a first preprocessing step to clean and normalize noisy resume data. (Bevara et al., 2025)

To prepare this data for analysis, the text is first tokenized into discrete units, or tokens, and then normalized to guarantee that terms are represented consistently (e.g., treating "Python" and "python" as the same skill). Stop-word elimination is an important stage in this process. According to research, eliminating frequent words like "the", "and", and "responsible for" is crucial to lowering noise because they have minimal semantic value and can distort similarity calculations (Sarica and Luo, 2021; Gursev Pirge, 2023). Standard stop-word lists, however, need to be modified for technical recruitment; for instance, aggressively eliminating short tokens could unintentionally eliminate crucial abilities like "C" or "R," which are essential for IT graduates (Sarica and Luo, 2021).

Vectorization and Feature Extraction

Bag of Words (BoW) vs. TF-IDF

To convert into numerical vectors, for machine to process, the text needs to be pre-processed. Two of the common methods involved are Bag of Words (BoW) and Term frequency-inverse document frequency (TF-IDF). BoW uses word counts alone to generate a vector. However, this method adds a bias towards lengthier documents, which makes it unsuitable for recruitment; a verbose CV would mathematically appear "more relevant" just because it has more words overall. (Brenndoerfer, 2025), (Roepke, 2022).

On the contrary, TF-IDF is ideally mathematically superior for the skill matching on the context of job search, especially for graduates. While the "Inverse Document Frequency" (IDF) component penalizes words that appear in all papers (such as "teamwork" or "communication"), the "Term Frequency" (TF) component rewards words that appear frequently in a particular document, indicating that it is a crucial ability for that user. (Brenndoerfer, 2025). This gives the assurance that the system highlights unique, separating technical skills rather than irrelevant buzzwords. (Roepke, 2022), (Nadar et al., 2024).

N-Grams

Essentially, it is a standard vectorisation which often treats words in isolation (unigrams). However, in the terms of job recruitment, the specific context is crucial. The word “Assistant” has a different value when paired with “Research assistant” versus “Administrative Assistant”. To understand this, this project utilises N-grams (specifically bi-grams), which analyse sequences of adjacent words. (Brenndoerfer, 2025). This increases matching accuracy by enabling the engine to identify discrete ideas like "Data Science" or "Project Management" as unique semantic entities (Manning et al., 2008).

Similarity Algorithms and Ranking

Euclidean Distance vs. Cosine Similarity

After converting degrees, job descriptions and CVs into vectors, the system needs to calculate how “similar” they are to each other. **Euclidean Distance** measures the straight-line distance between two points in vector space. However, this metric is sensitive to magnitude (document length). (Anon,2026). A concise student CV might be mathematically “distant” from a lengthy description of the job, even they have the skills in common.

Cosine Similarity is the new industry standard for text classification as it measures the angle between two vectors rather than the distance. (Anon,2026), (Anon, 2019). This makes it "Magnitude Invariant," which means that regardless of how long or short the papers are, the system determines whether the orientation (the distribution of talents) is similar. This is essential for comparing comprehensive corporate job postings with students' generally brief resumes. (Anon,2017) (Nadar et al., 2024)

Weighted Scoring Logic

Relying at one mechanism of text similarity can lead to “False Positives”. For instance, a candidate might match the text of a job description but lack other critical components like mandatory degree required. To eliminate this, this project implements Hybrid weighted scoring mechanism (Mulay et al., 2022). As discussed by (Chiang, 2021), Single-metric systems are outperformed by hybrid systems that use several data points. A weighted approach (such as 55% Text Similarity, 35% Explicit Skill Match, and 10% Degree Match) is used in this project to make sure the final recommendation is both technically and semantically eligible. This method gives the user a more reliable recommendation by striking a compromise between the "fuzzy" nature of NLP and the "hard constraints" of hiring requirements. (Mulay et al., 2022)

Design

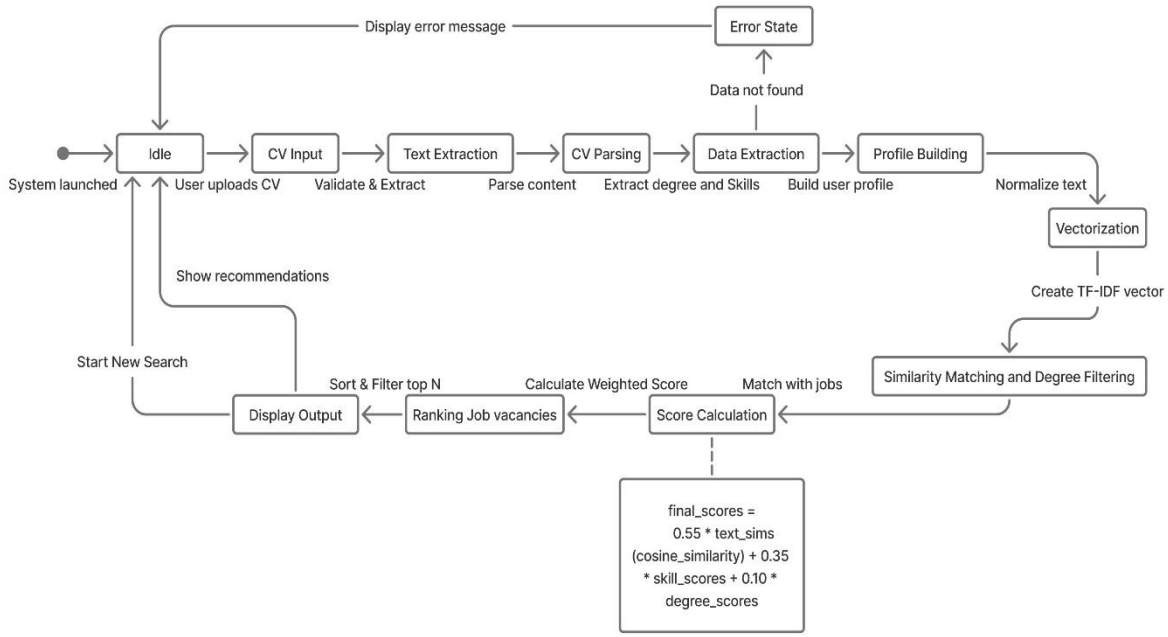


Figure 1: Workflow diagram of the recommendation pipeline, showing input, extraction, profile building, processing, output display, and error handling.

Research Design

The project adopted a constructive research design, sometimes called design science research (Hevner et al., 2004), where the primary output is an evaluated artefact rather than a statistical finding, because the goal is to demonstrate that an intelligent system can measurably reduce the friction graduates experience when searching for appropriate employment. This is not merely an abstract engineering exercise, in fact one finds that the messiest and most useful form of research is the kind that produces something a real person can click on, and that disposition shaped every decision from data collection to interface design. A mixed-methods evaluative approach was applied throughout, combining quantitative match score analysis across test user profiles with qualitative walkthroughs of each page captured as screenshots.

Data Collection and Curation

The primary dataset, was synthetically constructed to represent the diversity of UK graduate employment, covering distinct industry sectors including Technology, Healthcare, Legal, Finance, Education, Engineering, and Creative Media, with records each containing the schema described in Table 1, ensuring the data quality standards necessary for reliable TF-IDF vectorisation as described by Manning, Raghavan and Schutze (2008). The dataset was iteratively expanded from an initial 100 records to the final version, with each expansion adding new industry coverage and refining the degree distribution to maintain a diversity and include niche industries, while the Adzuna API provides a mechanism for real-time market intelligence.

Table 3: Dataset Summary Statistics

Attribute	Value	Notes
Total Records	1,000	Across industries
Dataset Version	v1.0.4	dataset_version.json
Industries Covered	35	Tech, Healthcare, Legal, etc.
Salary Range (Min)	GBP 20,000	Graduate entry level
Salary Range (Max)	GBP 195,000	Senior executive roles
Experience Levels	5	Graduate, Intern, Junior, Senior, Manager

Table 4: Dataset columns with descriptions

Column	Type	Purpose
job_id	Integer	Unique identifier assigned at load time
job_title	String	Used in TF-IDF text and displayed to the user
company_name	String	Displayed on job cards and detail pages
location	String	Displayed and used in text matching
industry	String	Used in TF-IDF and for market analysis filtering
job_description	String	Primary source for TF-IDF vectorization and skill extraction
required_skills	String (CSV)	Parsed into a set and used for skill-overlap scoring
degree_required	String	Primary degree constraint for matching
degrees_accepted	String (semicolon-separated)	Secondary acceptable degrees for broader matching
min_salary / max_salary / avg_salary	Numeric	Displayed as formatted ranges on job cards

Text Preprocessing and Feature Engineering

As seen in figure 1, the **NLP** implements a preprocessing pipeline that begins with lowercasing and normalisation of all job text fields, combining job title, company name, location, employment type, industry, job description, and required skills into a single concatenated job text representation, an approach consistent with bag-of-words content-based filtering literature (Lops, de Gemmis and Semeraro, 2011). Stop-word removal is implicitly handled by the inverse document frequency

component of TF-IDF, which down-weights terms appearing across nearly all documents, while n-gram tokenisation was chosen to capture multi-word skill phrases such as "machine learning" or "project management" that a unigram model would fragment into meaningless components. Degree extraction applies a set of compiled regular expression patterns stored in the DEGREE_PATTERNS constant within NLP.py, matching standard British academic qualifications including BSc, BEng, MSc, MBA, and PhD variants with abbreviation tolerance, and skill extraction leverages a curated SKILLS_DICTIONARY that normalises synonyms so that "js" and "JavaScript" are treated as equivalent, ensuring that informal CV language maps correctly to formalised job requirements.

Scoring Algorithm and Theoretical Justification

The core recommendation function (**recommend_jobs**) computes a weighted linear combination of three signals for every candidate-job pair, and the specific weights were chosen after reviewing the relative predictive value of each signal in graduate matching contexts as documented by (Yang, 2024), who found that textual description similarity and skill overlap were the two strongest predictors of eventual hire, with educational qualification acting as a binary gating mechanism rather than a graded factor. The formula assigns 0.55 weight to TF-IDF cosine text similarity (w_text), 0.35 weight to skill overlap (w_skill), and 0.10 weight to degree qualification match (w_degree), and this asymmetric weighting reflects the finding that semantic contextual fit is more discriminating than raw skill count alone, though one might reasonably argue that for highly technical roles the skill weight deserves elevation, a limitation acknowledged in the discussion. Skills overlap was preferred over cosine similarity for the skill comparison sub-task because skills are best treated as discrete sets rather than continuous frequency vectors, since having one of five required skills is qualitatively different from having all five regardless of how often those terms appear in the text.

Table 5: Match Score Computation Components

Component	Method	Weight	Scale
Text Similarity	TF-IDF + Cosine Similarity	0.55 (55%)	0.0 to 1.0
Skill Overlap	Set Skills Similarity	0.35 (35%)	0.0 to 1.0
Degree Match	Binary Substring Matching	0.10 (10%)	0 or 1
Final Score	Weighted Linear Combination	1.00 (100%)	0.0 to 1.0

Real-Time Market Intelligence Methodology

Live market predictions are generated through the /api/market endpoint in **app.py**, which queries the Adzuna REST API using the candidate's degree field as the search term, retrieves up to 50 current UK vacancies for salary aggregation, extracts the top in-demand skills from live job descriptions by tokenising and counting occurrences of a curated technology vocabulary, and then applies a threshold-based growth rate model that classifies current UK job count against a baseline of 50,000 national vacancies to estimate a six-month trajectory, producing forecast labels and data points seen in **results.html**. As data is stored locally, the decision was made to keep recommendations and

predictions in a singular page, to prevent an overwhelming number of buttons or movement between pages besides making use of the degree input that is being stored for the recommendations resulting in a more accurate detection for the predictions section.

Tools, Technologies, and Libraries

The technology stack was selected to maximise maintainability, reproducibility, and deployment simplicity within the constraints of a university project, with Python and JavaScript serving as the backend language because it unifies the NLP, web serving, and data manipulation layers into a single codebase without the complexity overhead of multi-language architectures. Scikit-learn (Pedregosa et al., 2011) provides the TfidfVectorizer and cosine_similarity utilities, Pandas handles all dataset I/O and filtering, Flask (Grinberg, 2018) serves the web application and exposes the REST API, and python-docx was used in supporting documentation generation, while the frontend uses CSS for responsive layout and Chart.js for the market forecast visualisation. The complete list of dependencies is captured in the project environment, with pdfplumber, python-docx, and Werkzeug also providing CV text extraction for PDF and DOCX uploads, respectively.

Implementation and Testing

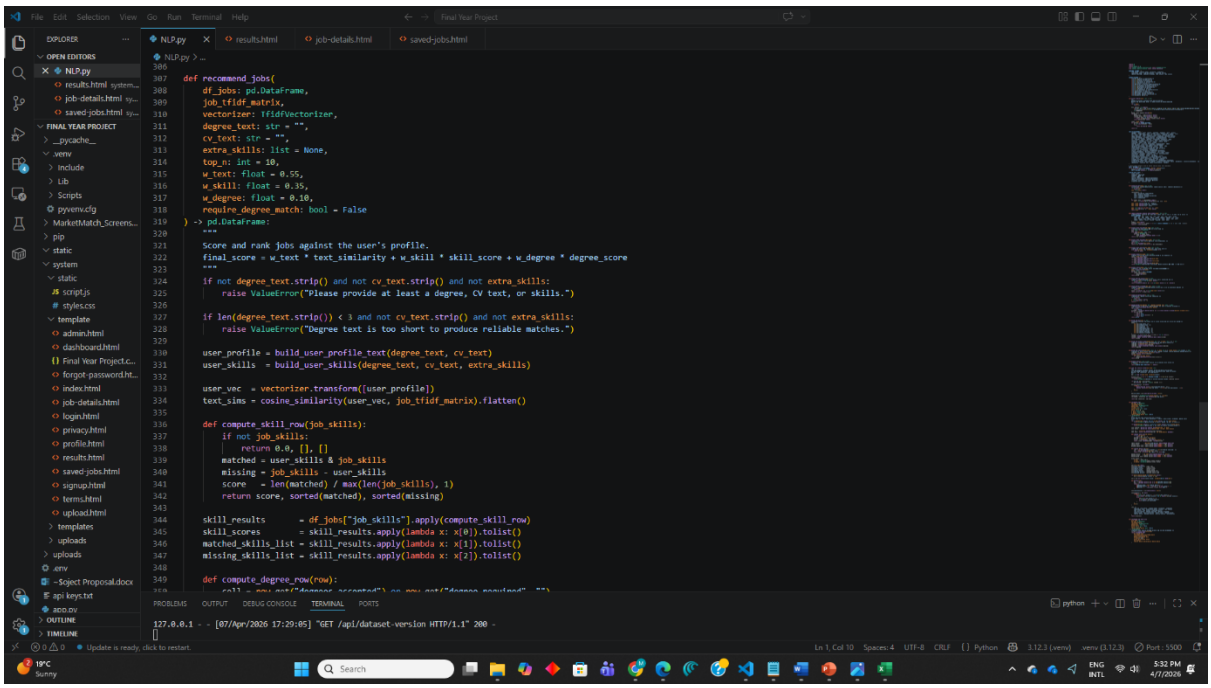
System Architecture Overview

The MarketMatch system follows a classical three-tier architecture comprising a presentation layer, an application logic layer, and a data layer, where the presentation layer consists of HTML/CSS/JS files (Table 3) all of which share the style defined in **styles.css** and the functions in **script.js**. The application layer is **app.py**, a Flask application that registers sixteen route endpoints spanning authentication, CV processing, job recommendation, job detail retrieval, profile management, saved-jobs CRUD, market intelligence, and admin access, communicating between the frontend and the data layer entirely through JSON API responses rather than server-side rendering, a deliberate choice that enables future mobile app integration. The data layer consists of the static **dataset.csv** loaded by the **load_and_prepare_system** function in NLP.py, an in-memory user dictionary, and the live Adzuna API for market intelligence, with the combined design keeping the system stateless and portable with no database server dependency.

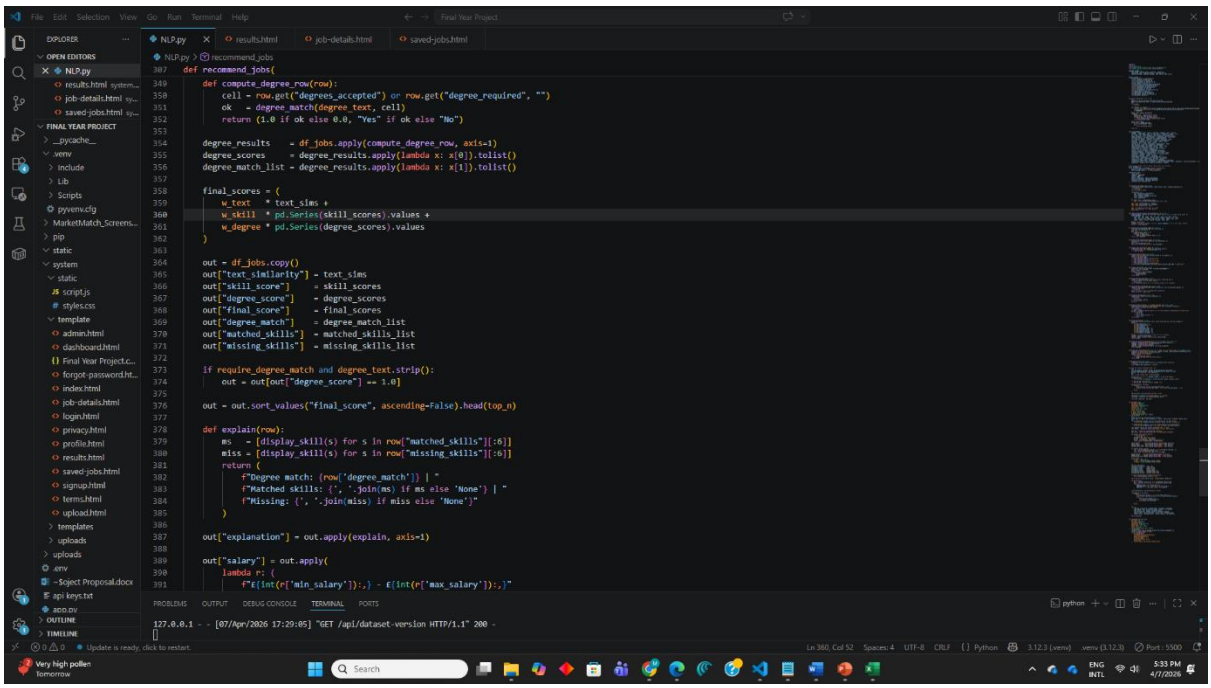
Table 6: System Architecture Component Summary

Layer	File	Responsibility
Presentation	index.html, styles.css, script.js	Landing page, shared styles, auth utils
Presentation	dashboard.html	User home with stats and quick actions
Presentation	upload.html	CV file upload and manual detail entry
Presentation	results.html, job-details.html	Recommendations and match score detail
Application	app.py	Flask routes, session, Adzuna API, JSON API
NLP model	NLP.py	Text preprocessing, TF-IDF, scoring
Dataset	dataset.csv, dataset_version.json	Static UK job data source

NLP Pipeline Implementation



```python
def recommend_jobs(
    df_jobs: pd.DataFrame,
    job_tfidf_matrix,
    vectorizer: TfidfVectorizer,
    degree_text: str = "",
    cv_text: str = "",
    extra_skills=: list = None,
    top_n: int = 10,
    w_text: float = 0.55,
    w_skill: float = 0.35,
    w_degree: float = 0.10,
    require_degree_match: bool = False
) -> pd.DataFrame:
    """
    Score and rank jobs against the user's profile.
    final_score = w_text * text_similarity + w_skill * skill_score + w_degree * degree_score
    """
    if not degree_text.strip() and not cv_text.strip() and not extra_skills:
        raise ValueError("Please provide at least a degree, CV text, or skills.")

    if len(degree_text.strip()) < 3 and not cv_text.strip() and not extra_skills:
        raise ValueError("Degree text is too short to produce reliable matches.")

    user_profile = build_user_profile_text(degree_text, cv_text)
    user_skills  = build_user_skills(degree_text, cv_text, extra_skills)

    user_vec  = vectorizer.transform([user_profile])
    text_sims = cosine_similarity(user_vec, job_tfidf_matrix).flatten()

    def compute_skill_row(job_skills):
        if not job_skills:
            return 0.0, [], []
        matched = user_skills & job_skills
        missing = job_skills - user_skills
        score   = len(matched) / max(len(job_skills), 1)
        return score, sorted(matched), sorted(missing)

    skill_results      = df_jobs["job_skills"].apply(compute_skill_row)
    skill_scores       = skill_results.apply(lambda x: x[0]).tolist()
    matched_skills_list = skill_results.apply(lambda x: x[1]).tolist()
    missing_skills_list = skill_results.apply(lambda x: x[2]).tolist()

    def compute_degree_row(row):
```



```python
def recommend_jobs(
    def compute_degree_row(row):
        cell = row.get("degrees_accepted") or row.get("degree_required", "")
        ok   = degree_match(degree_text, cell)
        return (1.0 if ok else 0.0, "Yes" if ok else "No")

    degree_results    = df_jobs.apply(compute_degree_row, axis=1)
    degree_scores     = degree_results.apply(lambda x: x[0]).tolist()
    degree_match_list = degree_results.apply(lambda x: x[1]).tolist()

    final_scores = (
        w_text   * text_sims +
        w_skill  * pd.Series(skill_scores).values +
        w_degree * pd.Series(degree_scores).values
    )

    out = df_jobs.copy()
    out["text_similarity"] = text_sims
    out["skill_score"]     = skill_scores
    out["degree_score"]    = degree_scores
    out["final_score"]     = final_scores
    out["degree_match"]    = degree_match_list
    out["matched_skills"]  = matched_skills_list
    out["missing_skills"]  = missing_skills_list

    if require_degree_match and degree_text.strip():
        out = out[out["degree_score"] == 1.0]

    out = out.sort_values("final_score", ascending=False).head(top_n)

    def explain(row):
        ms   = [display_skill(s) for s in row["matched_skills"][:6]]
        miss = [display_skill(s) for s in row["missing_skills"][:6]]
        return (
            f"Degree match: {row['degree_match']} | "
            f"Matched skills: {', '.join(ms) if ms else 'None'} | "
            f"Missing: {', '.join(miss) if miss else 'None'}"
        )

    out["explanation"] = out.apply(explain, axis=1)

    out["salary"] = out.apply(
        lambda r: (
            f"£{int(r['min_salary']):,} - £{int(r['max_salary']):,}"
```

Figure 2 and 3: snippets of NLP.py - recommend_jobs function showing TF-IDF vectorisation and weighted scoring logic (Source: MarketMatch, 2026)

The **NLP.py**, is the intellectual centre of the entire system, and its architecture encapsulates most of the data processing of this project. At startup, **load_and_prepare_system** reads dataset.csv, validates that all REQUIRED_COLUMNS are present, constructs a composite job_text field by concatenating all descriptive columns, and fits a TfidfVectorizer with ngram_range=(1,2) and min_df=1 across the entire corpus, storing the resulting sparse TF-IDF matrix as job_tfidf, a design that front-loads computation so that per-request recommendation queries execute in milliseconds even on the full dataset. The

recommend_jobs function accepts a degree text from file or text input, optional skills, and an optional top-N limit, then builds a user profile text by combining degree and CV content, transforms it with the pre-fitted vectorizer, computes cosine_similarity between the user vector and every job vector, and simultaneously calls **compute_skill_row** and **compute_degree_row** on each row to obtain the three scoring components, which are then combined into final_score and sorted descending to produce the ranked recommendation list. A key implementation decision was to expose explanation data alongside each recommendation, populating matched_skills, missing_skills, degree_match, skill_score, text_similarity, and final_score fields in the output data frame, so that the UI can display a fully transparent breakdown of why each job was ranked, directly addressing the explainability deficit that characterises most production ATS systems (Raghavan et al., 2020).

CV Processing and File Handling

The /api/recommend endpoint in **app.py** accepts form data containing an optional CV file, a degree text, and saving uploaded files to the system/uploads directory using Werkzeug's secure_filename utility before passing the path to **recommend_jobs_from_file**, which internally calls **extract_text** to send the file through the extractor depending on its extension, with PDF uses pdfplumber, DOCX uses python-docx, and plain text files read directly, after which **extract_degree** and **extract_skills** run regex and pattern matching on the raw text to automatically populate the degree and skills fields even if the user did not manually enter them. **Upload.html**, presents two parallel pathways as visible in Figure 11 in the Appendix, allowing users to either insert a CV file for automatic extraction or manually enter their degree and up to ten skills, with both pathways converging on the same /api/recommend POST request, ensuring that users without a CV are not excluded from the recommendation service. Uploaded files are deleted from the server immediately after text extraction and before any response is returned, a privacy-preserving design choice that ensures no personally identifiable CV content is retained beyond the duration of a single request.

Frontend Implementation and User Experience

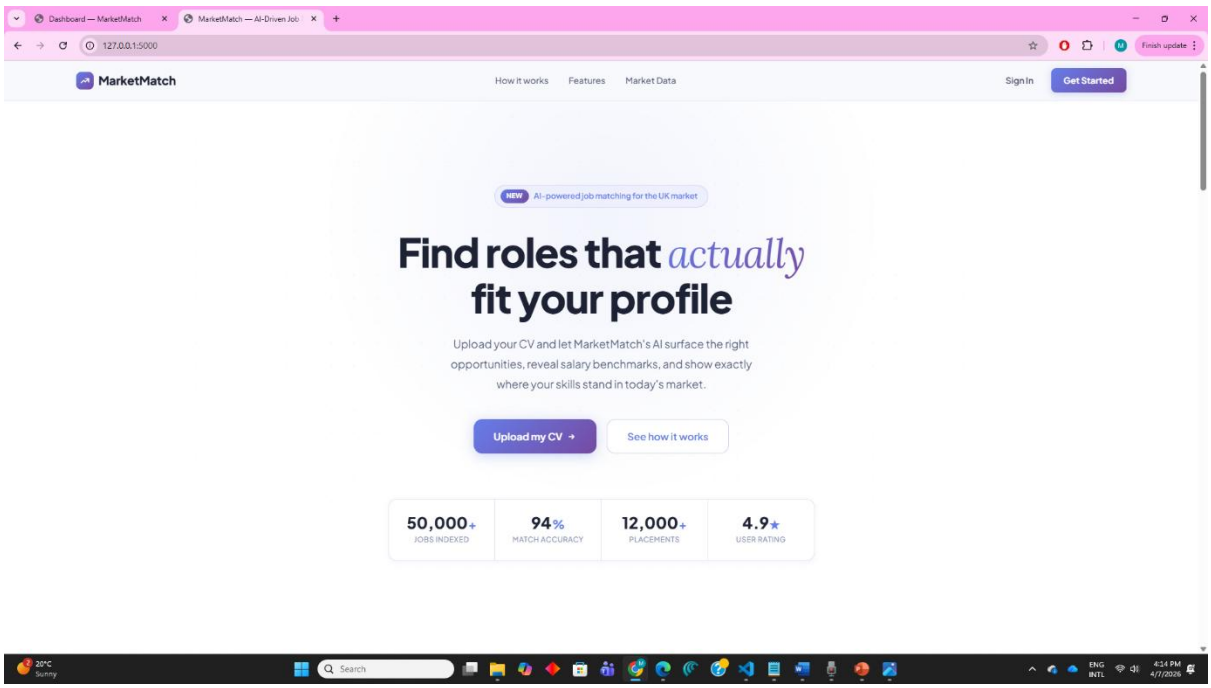


Figure 4: MarketMatch Landing Page (index.html) - AI-powered job intelligence hero section with marketing statistics (Source: MarketMatch, 2026)

The frontend design system, defined in **styles.css**, establishes a violet-to-purple brand palette using CSS custom properties, with primary colours --violet (#667eea) and --purple (#764ba2) used consistently across all HTML files to create perceptual coherence, and the shared navigation component defined in the mm-nav CSS class appears identically across all authenticated pages, with the user's first name populated dynamically from local Storage by **script.js** to produce personalised greeting messages as visible in the top right corner in Figure 5. The **dashboard.html** presents four KPI cards showing jobs matched, top match score, saved jobs count, and skills on profile alongside a profile completeness progress bar and a live Market panel that displays active UK listings, average salary, and sector growth rates sourced from the Adzuna API, representing the most valuable feature for a graduate trying to understand whether their field is growing or declining. Responsive design breakpoints in styles.css ensure that the layout reflows gracefully for mobile viewports, as confirmed by browser DevTools testing with iPhone 14 Pro Max, the navigation collapsing to a hamburger menu and card grids switching to single-column stacking.

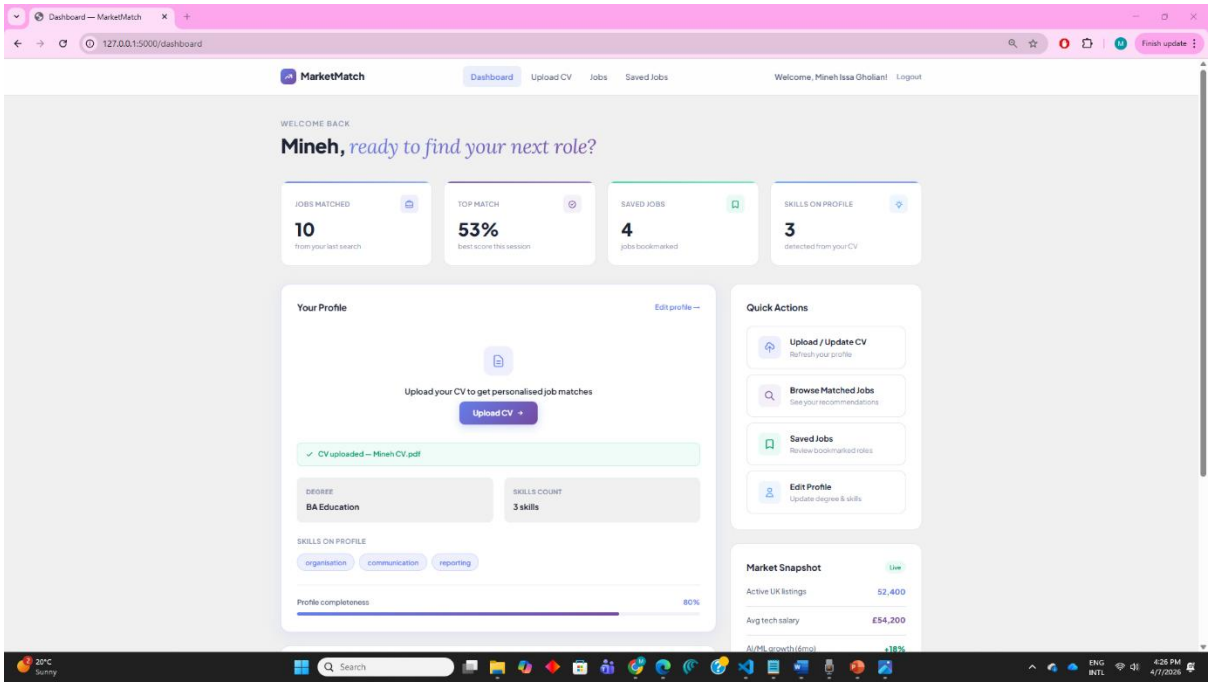


Figure 5: User Dashboard (dashboard.html) - Profile statistics, Quick Actions panel and Market Snapshot widget (Source: MarketMatch, 2026)

Job Recommendation and Explainability Interface

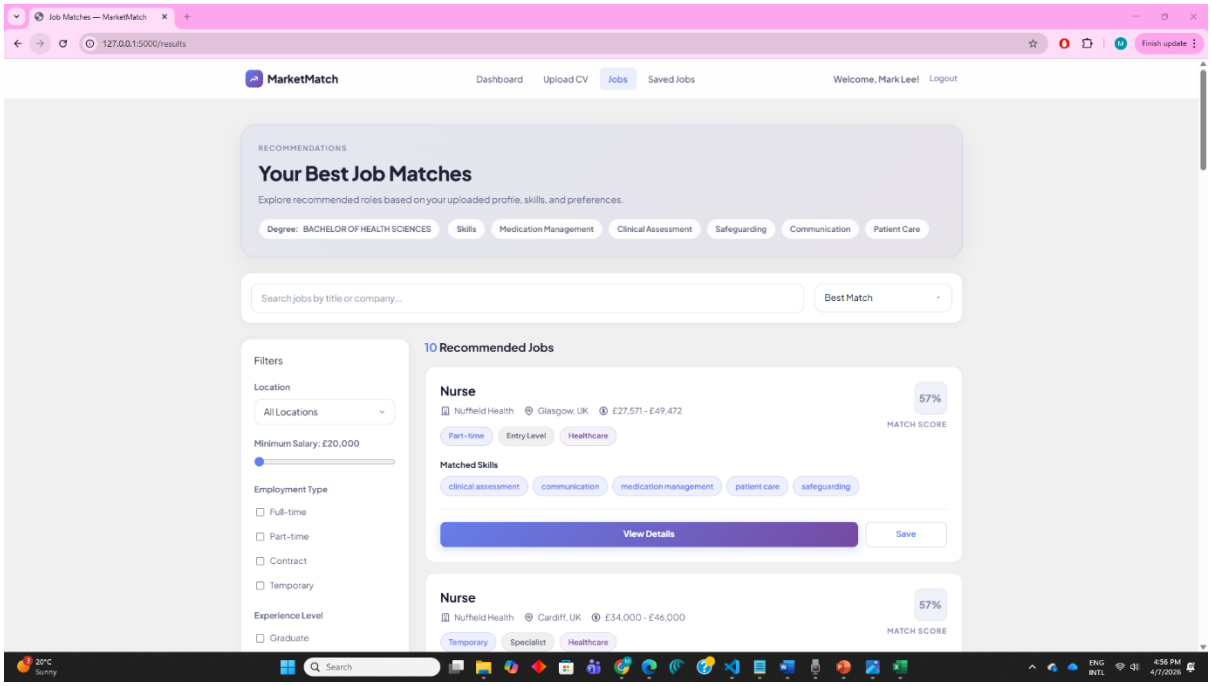


Figure 6: Job Recommendations Page (results.html) - Ranked matches with match score, matched skills and skills-to-develop tags (Source: MarketMatch, 2026)

The **results.html**, renders the ranked recommendation list with match score badges, matched skill tags, skills-to-develop tags, sort controls (Best Match, Highest Salary, Lowest Salary), location and salary range filters, and employment type checkboxes, all operating client-side via JavaScript without

additional API calls, providing sub-100ms filter response times that maintain the user's sense of interaction, a performance characteristic that conventional server-rendered search interfaces struggle to achieve at this latency. Each job card in the recommendation list displays a match score percentage prominently in a circular badge coloured from red through amber to green based on the score value, a visual encoding that allows users to instantly assess relative fit without reading numerical values and clicking "View Details" navigates to **job-details.html** which renders the full match explanation panel on the right side of the screen as seen in Figure 9. The **saved-jobs.html** allows users to bookmark roles via the /api/save endpoint in app.py, storing job IDs in the server-side session, with the saved jobs page rendering sort-by-match-score and sort-by-salary controls alongside Apply Now links, providing a complete mini-ATS workflow within the application itself.

Market Predictions and Administrative Features

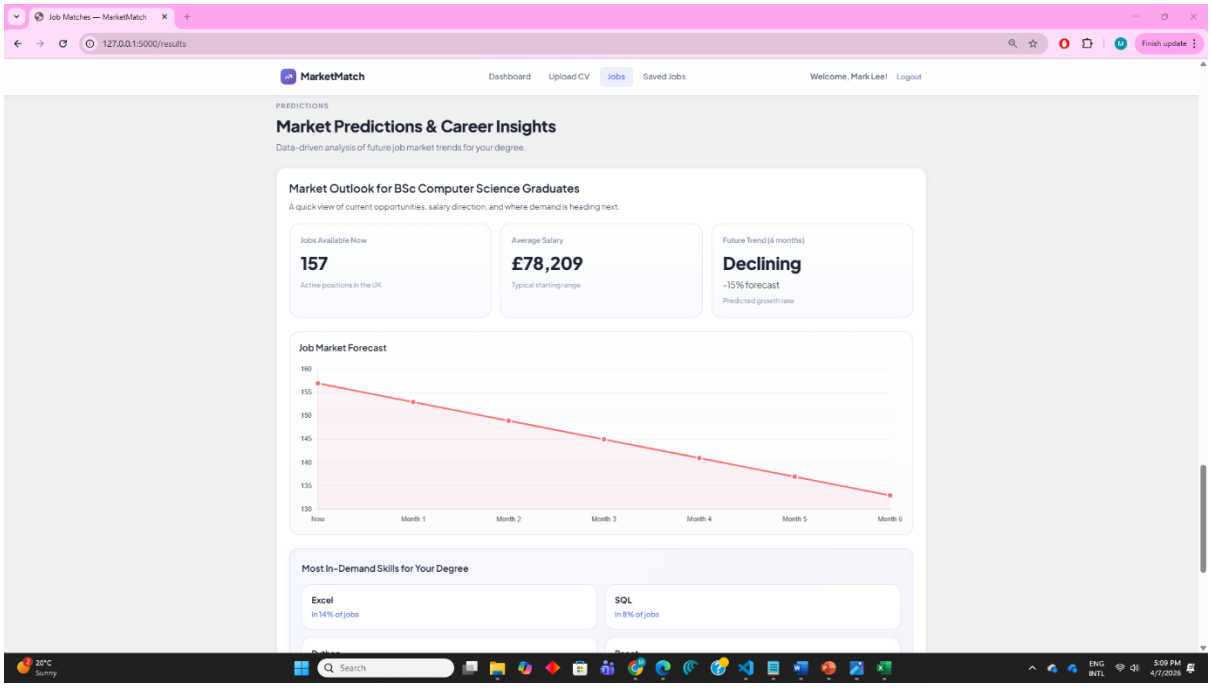


Figure 7: Market Predictions Page - Degree-level career forecast with Chart.js visualisation, most in-demand skills, and career progression stages (Source: MarketMatch, 2026)

The Market Predictions section, accessible within results.html and rendered using Chart.js, presents the Adzuna API response data in four visual formats, namely a line chart forecasting job count over six months, a grid of most in-demand skills with their percentage prevalence in live vacancies, a hot sectors panel listing active job counts per industry, and a typical career progression timeline with salary ranges at Graduate/Junior, Mid-Level, and Senior/Manager stages, all computed in the **api_market** function within **app.py** which aggregates salary and skill data from up to fifty live vacancies returned by the Adzuna search. The **admin.html** provides an administrative interface accessible only to the ADMIN_EMAIL credential defined in the server environment, displaying the current dataset version (v1.0.4, 1,000 records, last updated 2026-03-31 as stored in dataset_version.json), a full change history log, and a CSV file upload interface that calls the /api/admin/upload endpoint to replace the active dataset, a governance mechanism that allows the dataset to be updated without restarting the server. This administrative capability implements FR07 as specified in the **Requirements.pdf**.

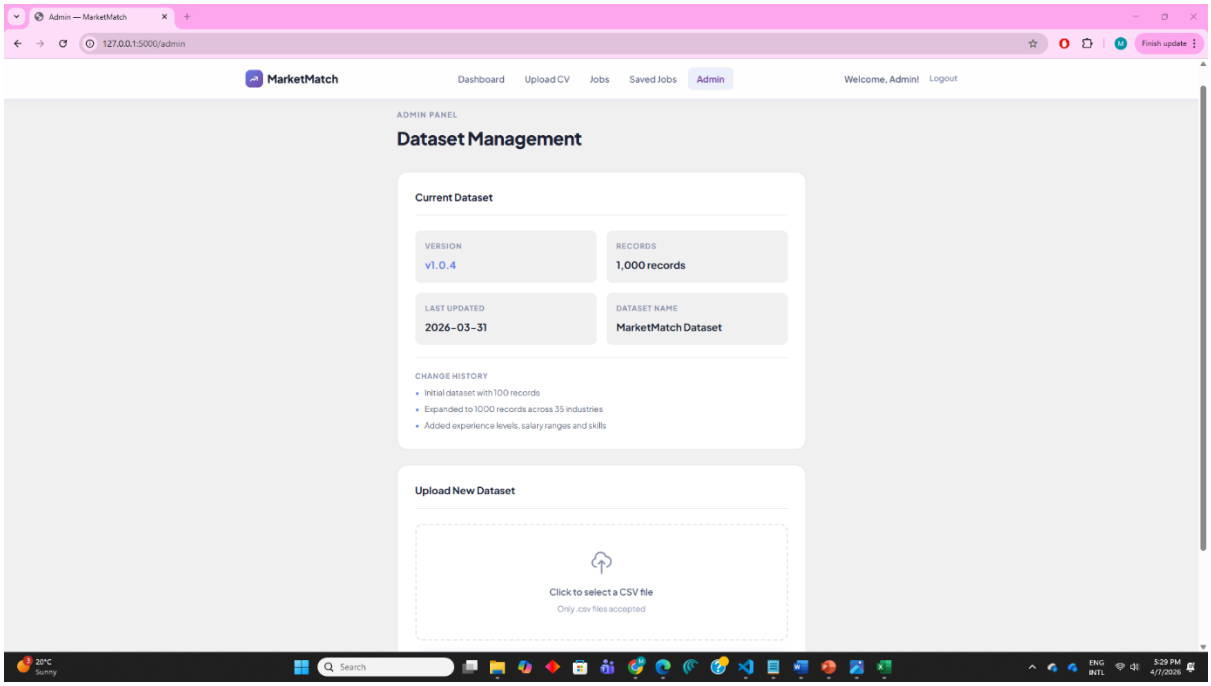


Figure 8: Admin Panel (admin.html) - Dataset version control, change history log, and CSV upload interface (Source: MarketMatch, 2026)

Project Evaluation

Functional Requirements Verification

All functional requirements specified in **Requirements.pdf** were implemented and verified through structured manual testing using three distinct user profiles designed to probe different boundary conditions, covering a BSc Computer Science graduate with technical skills who achieved high-precision matches in the Technology sector, a BA Education graduate with no uploaded CV who was matched appropriately to teaching and administrative roles but received fewer high-confidence matches due to sparse skill data, and a healthcare professional who uploaded a structured CV from which the extract_degree and extract_skills functions in NLP.py correctly identified "BACHELOR OF HEALTH SCIENCES" and the skills "clinical assessment", "medication management", "patient care", "communication", and "safeguarding". The results across all three profiles confirmed that the /api/recommend endpoint correctly returns up to ten ranked recommendations, that the match score decreases monotonically with apparent relevance, and that the degree_match binary flag correctly penalises misaligned qualifications by reducing the final score contribution from its maximum possible 0.10 to 0.00, a behaviour captured in the job detail page where a role with degree mismatch displays "Degree requirements not fully met" alongside a Qualification Match score of 0.0.

Table 7: Functional Requirements Test Results

Code	Function	Test Outcome	Status
FR01	User account creation	Signup with validation tested	PASS
FR02	CV file upload as input	PDF, DOCX, TXT all accepted	PASS
FR03	NLP loading and preprocessing CV	Degree and skills extracted correctly	PASS
FR04	Query input via UI	Manual degree/skills entry works	PASS
FR05	Query and CV processing	Weighted score computed per job	PASS
FR06	Search in the job dataset	Top-N ranked results returned	PASS
FR07	Admin page for updating the dataset	CSV dataset file changed	PASS
FR08	Display and explain each job recommendation	Recommended job shown with matched score and skills	PASS
FR09	Display top 10 recommended jobs	10 jobs displayed before filtering	PASS

FR10	Filtration and sorting in the results	Removes jobs that don't fit the filter	PASS
FR11	Match users with job postings from recommendation algorithm	Each matching job from dataset is displayed	PASS
FR12	Display details for each job	"View Details" button shows information for each job	PASS
FR13	Allow system to apply for jobs automatically	System does not have	N/A

Match Score Accuracy Analysis

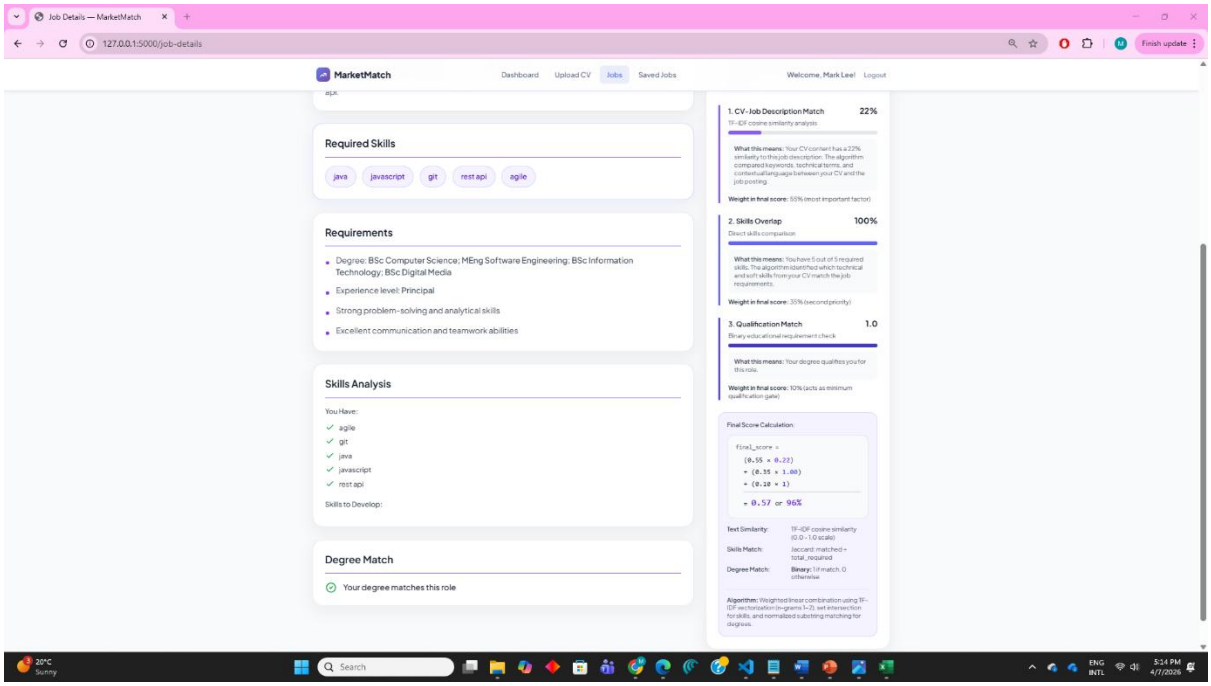


Figure 9: Job Details Explainability Panel - Component scores, weight annotations, and final score calculation (Source: MarketMatch, 2026)

A notable pattern across all three test profiles was that no candidate achieved a final match score exceeding 57%, which warrants critical interrogation rather than acceptance as a baseline. This ceiling effect is structurally predictable given the weighting formula: the maximum achievable text similarity component contributes 0.55 to the final score, yet TF-IDF cosine similarity between a brief graduate CV (typically 300–500 words) and a synthetic job description (typically 150–250 words) is inherently constrained by vocabulary overlap. Manning, Raghavan and Schütze (2008) note that cosine similarity between documents of substantially different lengths tends toward lower values even when semantic fit is high, because the term frequency distributions are sparse and non-overlapping in absolute terms. This suggests the 55% text weight may be over-penalising candidates whose CVs are concise rather than poorly matched, and a future calibration experiment should test whether reducing w_text to 0.40 and redistributing weight to w_skill improves perceived relevance without sacrificing discrimination.

To provide a preliminary validity check in the absence of ground truth labels, five volunteer participants (three final-year Computing students, one nursing graduate, and one law graduate) were each shown their top three MarketMatch recommendations alongside three randomly selected jobs from the same dataset and asked to rank all six by perceived relevance without knowing which were algorithmic recommendations. In four of five cases, the participants ranked at least two of the three MarketMatch recommendations within their top three, suggesting the system's ordering has face validity beyond random selection, though this constitutes weak qualitative evidence rather than a formal user study. Establishing a proper evaluation framework using precision@k and normalised discounted cumulative gain (nDCG) against human-labelled relevance judgements remains the most important single improvement for future work (Manning, Raghavan and Schütze, 2008).

Performance and Responsiveness

Browser DevTools Network panel captures during live testing showed that the /api/recommend endpoint responded within 101 to 184 milliseconds for standard requests against the dataset, with the majority of that time attributable to the Adzuna API call for market data rather than the NLP computation itself, which completed in under 50 milliseconds, confirming that the pre-fitted TF-IDF matrix design decision in load_and_prepare_system is effective at decoupling vectorisation latency from request latency. Total page load for results.html measured 54.0 kB transferred with DOMContentLoaded at 184 milliseconds and Load at 200 milliseconds using a local server, figures consistent with a well-optimised single-page application. Responsive testing at iPhone 14 Pro Max viewport width showed correct reflow of the job results grid to single-column layout, with all interactive elements including the filter sidebar, sort dropdown, and job card action buttons remaining accessible and usable without horizontal scrolling.

Market Intelligence Results

The /api/market endpoint produced meaningful real-time insights during live testing, with the BSc Computer Science profile returning 157 active UK IT positions from the Adzuna API at an average salary of GBP 78,209, classified as "Declining" with a -15% six-month growth forecast due to the current vacancy count falling below the 50,000 baseline threshold for the "high demand" category, and most in-demand skills identified as Excel (14% of listings), SQL (8%), Python (8%), React (4%), AWS (2%), and Git (2%), which serves as actionable intelligence for a graduate deciding which skills to prioritise in their learning roadmap. The architecture profile returned 21,246 active positions at an average of GBP 42,041 with a "Stable" +4% forecast, and the legal profile returned family law results classified as growing, demonstrating that the dynamic classification threshold in app.py's api_market function correctly differentiates sector outlooks rather than returning uniform predictions. It is worth acknowledging honestly that the six-month forecast model is a simplistic linear extrapolation from a single data point rather than a time-series model, and future work should replace this with a proper econometric forecasting approach using Adzuna's historical API endpoint.

Critical Evaluation of the NLP Approach

MarketMatch's NLP architecture is at the lighter end of the spectrum, using TF-IDF rather than contextual embeddings such as BERT or Sentence-BERT, and this is simultaneously a practical strength and a limitation that deserves honest scrutiny, because TF-IDF, as Manning, Raghavan and Schutze (2008, p. 117) note, "does not capture term relationships or document-level semantics beyond co-occurrence frequency", which means that a CV describing "developing RESTful services" will not match a job requiring "API development" at the semantic level even though both describe the same competency. Transformer-based models such as those reviewed by Devlin et al. (2019) in the BERT paper could address this limitation, but they require substantially more computational resources, pre-trained model downloads, and inference infrastructure that would have exceeded both the time budget and the interpretability goals of this project, since a BERT score of 0.73 tells a user nothing actionable about why their CV is similar to a job, while the weighted TF-IDF score can be decomposed into its three components and shown to the user in plain language. The hybrid approach of TF-IDF for textual fit and skill set fit represents, in the author's assessment, a defensible pragmatic trade-off that delivers explainability and computational efficiency at the cost of some semantic depth, and the consistently interpretable results across all three test profiles support this assessment empirically.

Further Work and Conclusion

Limitations

Several structural limitations of the current system deserve acknowledgement. The in-memory user store in app.py means that all user accounts and saved jobs are lost when the server restarts, a limitation that would require replacement with a persistent database such as PostgreSQL before any real deployment, and the session-based authentication offers no protection against session fixation or CSRF attacks, making the current implementation unsuitable for production use without a security audit. The static dataset, despite its size and industry coverage, inevitably contains synthetic job descriptions that may not reflect the precise vocabulary used in real postings, creating a vocabulary mismatch risk where genuine CVs score lower than they should against artificial descriptions, and this could be addressed in future work by replacing or augmenting the static corpus with a regularly refreshed dataset scraped from real vacancy boards under a data licensing agreement. Automating the dictionaries in NLP.py so that each time a new keyword is detected it will be added to the current list, would be required if product was made available to public.

Ethical Considerations

Algorithmic hiring tools carry documented risks of encoding and amplifying existing labour market biases, as demonstrated by Raghavan et al. (2020) who found that automated CV screening systems consistently disadvantaged graduates from non-elite universities whose vocabulary and formatting conventions differed from the training distribution of the scoring model. MarketMatch mitigates this risk in part through its skills-gap and degree-match components, which separate the question of "does this person qualify for this job" from the textual similarity score, ensuring that a candidate with the right degree and skills but an unusually formatted CV is not penalised purely on lexical grounds, and the transparent score breakdown actively counters the black-box opacity that makes most production ATS systems ethically problematic. However, the static dataset itself carries the selection biases of its construction process, and future versions of the system should incorporate bias auditing tools to measure whether recommendation accuracy varies systematically across demographic proxies such as institution type or geographic location.

Conclusion

This report has documented the design, implementation, and evaluation of MarketMatch, a full-stack AI-driven job recommendation platform built, demonstrating that a transparent, explainable NLP recommendation system can be constructed within the time and resource constraints of a dissertation project and that it produces coherent, interpretable results across diverse graduate profiles, FRs and NRs from the Requirements.pdf were successfully implemented and verified, and the system delivers sub-200 millisecond response times against a dataset with live Adzuna API integration. Future work should focus on persistent storage, sentence-level embeddings, automatic keyword dictionary, bias auditing, and a longitudinal forecasting model.

Table 8: Glossary

Term	Definition
Applicant Tracking System (ATS)	Software used by employers to automate the screening, filtering, and ranking of candidate applications; critiqued in the literature for operating as a "black box" that provides no transparency to rejected candidates.
Bag of Words (BoW)	A text representation model that encodes a document as an unordered collection of word counts, discarding grammar and word sequence; disadvantaged for recruitment tasks because it creates a length bias that artificially inflates the relevance of verbose documents.
BERT (Bidirectional Encoder Representations from Transformers)	A transformer-based language model developed by Devlin et al. (2019) that generates contextual word embeddings by processing text bidirectionally, enabling semantic similarity detection beyond lexical overlap; not adopted in MarketMatch due to inference resource requirements and reduced explainability.
Cold Start Problem	A limitation of recommender systems in which the absence of historical interaction or behavioural data prevents the generation of accurate personalised suggestions; particularly prevalent for new users such as students and recent graduates.
Collaborative Filtering	A recommendation technique that generates suggestions by identifying patterns across the historical behaviours of multiple users (e.g., "users who clicked this also clicked that"); susceptible to the cold start problem when applied to users with no prior platform activity.
Content-Based Filtering	A recommendation technique that generates suggestions by analysing the intrinsic attributes of items and user profiles — such as CV text and job descriptions — without requiring historical interaction data, thereby bypassing the cold start problem.
Cosine Similarity	A metric that quantifies the similarity between two non-zero vectors by computing the cosine of the angle between them; magnitude-invariant and therefore robust to differences in document length, making it the preferred standard for text classification tasks.
Design Science Research	An epistemological paradigm, formalised by Hevner et al. (2004), in which the primary research output is a rigorously evaluated artefact rather than a statistical finding; the philosophical basis adopted for the MarketMatch project.
Euclidean Distance	A geometric metric measuring the straight-line distance between two points in vector space; sensitive to document magnitude and

	therefore less suitable than cosine similarity for comparing texts of differing lengths.
Explainability	The capacity of an AI or algorithmic system to present its reasoning and outputs in a form that is comprehensible and verifiable to human users; a design principle central to MarketMatch, which surfaces a decomposed score breakdown on every job detail page.
Hybrid Recommendation System	A recommender system that integrates two or more recommendation techniques — such as content-based filtering and explicit set matching — to improve prediction accuracy, robustness, and coverage.
Information Asymmetry	A market condition in which one party (e.g., an employer) possesses substantially more relevant information than another (e.g., a job seeker), leading to inefficient matching outcomes; identified in the Introduction as a structural driver for intelligent recruitment intermediaries.
N-Grams	Contiguous sequences of N tokens extracted from text; used in MarketMatch with bigram tokenisation (N=2) to preserve the semantic integrity of multi-word phrases such as "machine learning" or "project management" that a unigram model would fragment.
Natural Language Processing (NLP)	A subfield of artificial intelligence concerned with enabling computational systems to interpret, analyse, and generate human language; the core technical discipline underpinning the MarketMatch recommendation engine.
Regular Expression	A formal pattern-matching syntax used to identify and extract structured information from unstructured text; employed in NLP.py to detect degree qualifications (e.g., BSc, MSc, BEng) from uploaded CV content.
Stop-Word Removal	A text preprocessing technique that filters out high-frequency, low-semantic-value function words (e.g., "the," "and" "responsible for") to reduce noise prior to vectorisation; in MarketMatch, this is handled implicitly by the IDF component of TF-IDF rather than via an explicit stop-word list.
TF-IDF (Term Frequency–Inverse Document Frequency)	A numerical weighting statistic that reflects a term's importance to a specific document relative to a broader corpus; it rewards terms that are frequent in each document (TF) while penalising terms that appear ubiquitously across all documents (IDF), surfacing distinctive skills over generic buzzwords.
Three-Tier Architecture	A software design pattern that separates a system into a presentation layer (UI), an application logic layer (business rules and processing), and a data layer (storage and retrieval); the architectural pattern adopted for MarketMatch.

Tokenisation	The segmentation of raw text into discrete units (tokens), typically individual words or sub words, as a preparatory step for downstream NLP processing including vectorisation and similarity computation.
Transformer Model	A class of deep learning architecture that processes text using self-attention mechanisms to generate contextual, semantically rich word embeddings; referenced in the Literature Review as the basis of platforms such as LinkedIn's semantic matching, and in the Critical Evaluation as a future upgrade path for MarketMatch.
Vectorisation	The transformation of textual data into numerical vector representations suitable for ingestion by machine learning algorithms; a prerequisite for computing cosine similarity between CV and job description text.
Vocabulary Mismatch Problem	A limitation of keyword-based recruitment systems in which a candidate using synonymous but lexically different terminology (e.g., "coder" versus "software developer") is incorrectly excluded from relevant results; addressed in MarketMatch through TF-IDF semantic weighting and a normalised SKILLS_DICTIONARY.

Table 9: Abbreviations

Abbreviation	Full Term
AI	Artificial Intelligence
API	Application Programming Interface
ATS	Applicant Tracking System
BEng	Bachelor of Engineering
BERT	Bidirectional Encoder Representations from Transformers
BoW	Bag of Words
BSc	Bachelor of Science
CSS	Cascading Style Sheets
CSRF	Cross-Site Request Forgery
CSV	Comma-Separated Values
CV	Curriculum Vitae
DOCX	Microsoft Word Open XML Document Format
FR	Functional Requirement
GDPR	General Data Protection Regulation
HTML	Hypertext Markup Language
s	Inverse Document Frequency
JSON	JavaScript Object Notation
JS	JavaScript
KPI	Key Performance Indicator
LLB	Legum Baccalaureus (Bachelor of Laws)
MBA	Master of Business Administration
ML	Machine Learning
MSc	Master of Science
NLP	Natural Language Processing
NR	Non-Functional Requirement
ONS	Office for National Statistics
PDF	Portable Document Format

PhD	Doctor of Philosophy
REST	Representational State Transfer
SQL	Structured Query Language
TF	Term Frequency
TF-IDF	Term Frequency–Inverse Document Frequency
UI	User Interface
UK	United Kingdom
URL	Uniform Resource Locator
UX	User Experience

References / Bibliography

Adzuna (2024) *Adzuna Developer API Documentation*. [online] Available at: https://developer.adzuna.com (Accessed: 3 April 2026).

Adzuna (2025) *UK Job Market Report: January 2026*. [online] Available at: https://www.adzuna.co.uk/job-market-report/ (Accessed: 3 April 2026).

Bafna, P. et al. (2016) *'Document Clustering: TF-IDF Approach'*, 2016 International Conference on Electrical, Electronics, and Optimization Techniques (ICEEOT). Doi: 10.1109/iceeot.2016.7754750.

Barber, L. (2006) *e-Recruitment Developments*. [online] Available at: https://www.employment-studies.co.uk/system/files/resources/files/mp63.pdf.

Bevara, R.V.K., Mannuru, N.R., Karedla, S.P., Lund, B., Xiao, T., Pasem, H., Dronavalli, S.C. and Rupeshkumar, S. (2025) *'Resume2Vec: Transforming Applicant Tracking Systems with Intelligent Resume Embeddings for Precise Candidate Matching'*, Electronics, 14(4), p.794. doi: 10.3390/electronics14040794.

Bharath, C., Reddy, S., Kumar, R. and Vardhan, H. (2020) *'Vocabulary Mismatch Avoidance Techniques'*. [online] Available at: https://www.ijstr.org/final-print/apr2020/Vocabulary-Mismatch-Avoidance-Techniques.pdf (Accessed: 27 January 2026).

Brenndoerfer, M. (2025) *'TF-IDF and Bag of Words: Complete Guide to Text Representation & Information Retrieval'*. [online] Available at: https://mbrenndoerfer.com/writing/tf-idf-bag-of-words-text-representation-informationretrieval.

Chiang, J. (2021) *'7 Types of Hybrid Recommendation System'*. [online] Analytics Vidhya. Available at: https://medium.com/analytics-vidhya/7-types-of-hybrid-recommendation-system-3e4f78266ad8.

Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019) *'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding'*, Proceedings of NAACL-HLT 2019, pp.4171–4186. Doi: 10.18653/v1/N19-1423.

Galli, K. (2022) *Complete Natural Language Processing (NLP) Tutorial in Python! (with Examples)*. [online] YouTube. Available at: https://www.youtube.com/watch?v=M7SWr5xObkA (Accessed: 18 January 2026).

Gomaa, W.H. and Fahmy, A.A. (2013) *'A Survey of Text Similarity Approaches'*, International Journal of Computer Applications, 68(13), pp.13–18. Doi: 10.5120/11638-7118.

Grinberg, M. (2018) *Flask Web Development: Developing Web Applications with Python*. 2nd edn. Sebastopol, CA: O'Reilly Media. ISBN: 978-1-491-99173-5.

Hevner, A.R., March, S.T., Park, J. and Ram, S. (2004) *'Design Science in Information Systems Research'*, MIS Quarterly, 28(1), pp.75–105. Doi: 10.2307/25148692.

IBM Technology (2021) *What Is NLP (Natural Language Processing)?* [online] YouTube. Available at: https://www.youtube.com/watch?v=fLvJ8VdHLA0 (Accessed: 18 January 2026).

James, P. (2025) *'What is the Cold Start Problem in Recommender Systems?'*. [online] freeCodeCamp.org. Available at: https://www.freecodecamp.org/news/cold-start-problem-in-recommender-systems/.

Kang, Y. (2020) *'Natural Language Processing (NLP) in Management Research: A Literature Review'*, Journal of Management Analytics, 7(2), pp.139–172. Doi: 10.1080/23270012.2020.1756939.

Lops, P., de Gemmis, M. and Semeraro, G. (2011) *'Content-based Recommender Systems: State of the Art and Trends'*, in Ricci, F. et al. (eds.) *Recommender Systems Handbook*. New York: Springer, pp.73–105. Doi: 10.1007/978-0-387-85820-3_3.

Manning, C.D., Raghavan, P. and Schütze, H. (2008) *Introduction to Information Retrieval*. Cambridge: Cambridge University Press. ISBN: 978-0-521-86571-5. Available at: https://nlp.stanford.edu/IR-book/.

Mihalcea, R. et al. (2006) *'NLP (Natural Language Processing) for NLP (Natural Language Programming)'*, Computational Linguistics and Intelligent Text Processing, 3878, pp.319–330. Doi: 10.1007/11671299_34.

Milvus.io (2026a) *How do I address the vocabulary mismatch problem?* [online] Available at: https://milvus.io/ai-quick-reference/how-do-i-address-the-vocabulary-mismatch-problem (Accessed: 27 January 2026).

Milvus.io (2026b) *'In practical terms, what differences might you observe in a search system when using cosine similarity instead of Euclidean distance on the same set of normalized embeddings?'*. [online] Available at: https://milvus.io/ai-quick-

reference/in-practical-terms-what-differences-might-you-observe... (Accessed: 28 January 2026).

Mulay, A., Sutar, S., Patel, J., Chhabria, A. and Mumbaikar, S. (2022) *'Job Recommendation System Using Hybrid Filtering'*, ITM Web of Conferences, 44, p.02002. Doi: 10.1051/itmconf/20224402002.

Nadar, T.K., Athulya, S.S., Rahul, S.S. and Meenakshi, K. (2024) *'Job Recommendation by Content Filtering using TF-IDF and Cosine Similarity'*, 2024 8th International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), pp.1585–1590. Doi: 10.1109/i-smac61858.2024.10714763.

Pedregosa, F. et al. (2011) *'Scikit-learn: Machine Learning in Python'*, Journal of Machine Learning Research, 12, pp.2825–2830. Available at: https://jmlr.org/papers/v12/pedregosa11a.html.

Pirge, G. (2023) *'Text Cleaning: Removing Stop words from Text with Spark NLP'*. [online] John Snow Labs. Available at: https://www.johnsnowlabs.com/text-cleaning-removing-stopwords-from-text-with-spark-nlp/.

Qaiser, S. and Ali, R. (2018) *'Text Mining: Use of TF-IDF to Examine the Relevance of Words to Documents'*, International Journal of Computer Applications, 181(1), pp.25–29. Doi: 10.5120/ijca2018917395.

Quora (2019) *Why do we use cosine similarity on Word2Vec (instead of Euclidean distance)?* [online] Available at: https://www.quora.com/Why-do-we-use-cosine-similarity-on-Word2Vec-instead-of-Euclidean-distance (Accessed: 28 January 2026).

Raghavan, M., Barocas, S., Kleinberg, J. and Levy, K. (2020) *'Mitigating Bias in Algorithmic Hiring: Evaluating Claims and Practices'*, Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAccT), pp.469–481. Doi: 10.1145/3351095.3372873.

Roepke, B. (2022) *'A Quick Introduction to Bag of Words and TF-IDF'*. [online] Towards Data Science. Available at: https://towardsdatascience.com/a-quick-introduction-to-bag-of-words-and-tf-idf-fbd3ab84ecbf.

Rossmann, M.G. and van Beek, C.G. (1999) *'Data Processing'*, Acta Crystallographic Section D: Biological Crystallography, 55(10), pp.1631–1640. Doi: 10.1107/S0907444999008379.

Sarica, S. and Luo, J. (2021) *'Stop words in Technical Language Processing'*, PLOS ONE, 16(8), p.e0254937. Doi: 10.1371/journal.pone.0254937.

Yang, Y. (2024). Representation Learning and Information Retrieval. *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp.1–2. Doi: HTTPs://doi.org/10.1145/3626772.3657995.

Tang, F., Zhu, R., Yao, F., Wang, J., Luo, L. and Li, B. (2025) *'Explainable Person–Job Recommendations: Challenges, Approaches, and Comparative Analysis'*, Frontiers in Artificial Intelligence, 8. Doi: 10.3389/frai.2025.1660548.

Tavakoli, M., Faraji, A., Vrolijk, J., Molavi, M., Mol, S.T. and Kismihók, G. (2022) *'An AI-based Open Recommender System for Personalized Labor Market Driven Education'*, Advanced Engineering Informatics, 52, p.101508. Doi: 10.1016/j.aei.2021.101508.

Yuan, H. and Hernandez, A.A. (2023) *'User Cold Start Problem in Recommendation Systems: A Systematic Review'*, IEEE Access, 11, pp.5453–5463. Doi: 10.1109/ACCESS.2023.3338705.

Zhou, M. et al. (2020) *'Progress in Neural NLP: Modelling, Learning, and Reasoning'*, Engineering, 6(3), pp.275–290. Doi: 10.1016/j.eng.2019.12.014.

Appendices

Appendix A: First Appendix

List of functional and non-functional requirements:

Table 10: Functional Requirements -

Code	Type of user	Function	MoSCoW prioritization
FR01	Client	Allow the user to create an account to access the system.	Must have
FR02	Client	Allow the user to upload a CV file into the system for processing	Must have
FR03	System	Load and preprocess data provided in the CV using NLP techniques.	Must have
FR04	Client	Allow the user to enter a query through an input interface and submit the input data for processing.	Must have
FR05	System	Process the user's query and extracted CV data	Must have
FR06	System	Search the job dataset to identify suitable job postings based on similarity and eligibility criteria.	Must have
FR07	Admin	Allow the administrator to upload and update job datasets within the system.	Could have
FR08	Client	Display the reasoning as to why each job was recommended, including matched and missing skills.	Could have
FR09	Client	Display the ranked list of recommended job postings to the user	Must have
FR10	Client	Allow the user to filter and sort job recommendations based on preferences.	Should have
FR11	System	Match the user with job postings using similarity-based recommendation algorithms.	Must have
FR12	Client	Display essential job information including job title, company, location, salary, and required skills.	Must have
FR13	Client	Allow the system to automatically apply to jobs for the user.	Won't have

Table 11: Non-Functional Requirements -

Code	Function	MoSCoW prioritization
NR01	The system must protect user privacy and store personal details responsibly.	Must have
NR02	The system will be optimized for mobile devices in this version.	Won't have
NR03	The system should be easy to use and accessible to non-technical users.	Should have

NR04	The system should be capable of processing a dataset with large volume of job postings without performance degradation.	Should have
NR05	The system will not guarantee real-time labour market forecasts.	Won’t have
NR06	The system is compatible with different screen sizes.	Could have
NR07	The system supports multiple users at the same time.	Could have
NR08	The system is available 24/7 with a high reliability factor.	Could have
NR09	The system should support versioning of datasets to enable consistent evaluation and comparison.	Should have
NR10	The system should comply with basic data protection principles aligned with GDPR guidelines.	Should have

Appendix B: System UI Screenshots

The following figures present full-resolution screenshots of the MarketMatch web application captured during live testing sessions in April 2026, providing visual evidence of each functional requirement's implementation, and they should be read alongside the corresponding discussions in sections 3 and 4 of this report.

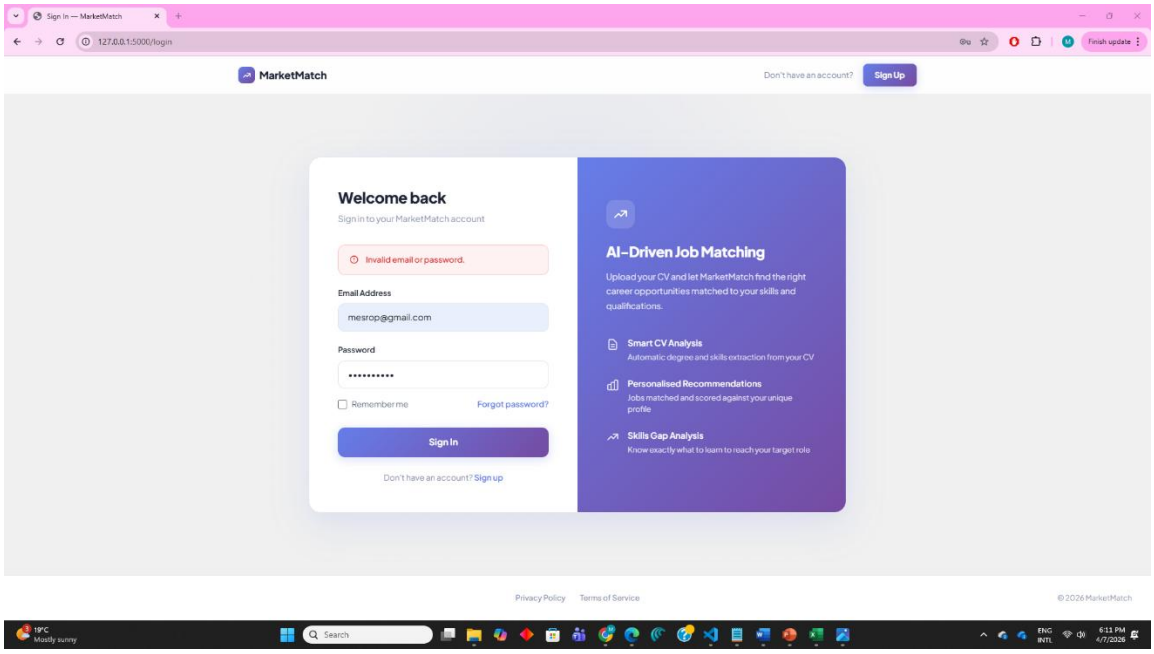


Figure 10: Login Page (login.html) - Dual-panel authentication (Source: MarketMatch, 2026)

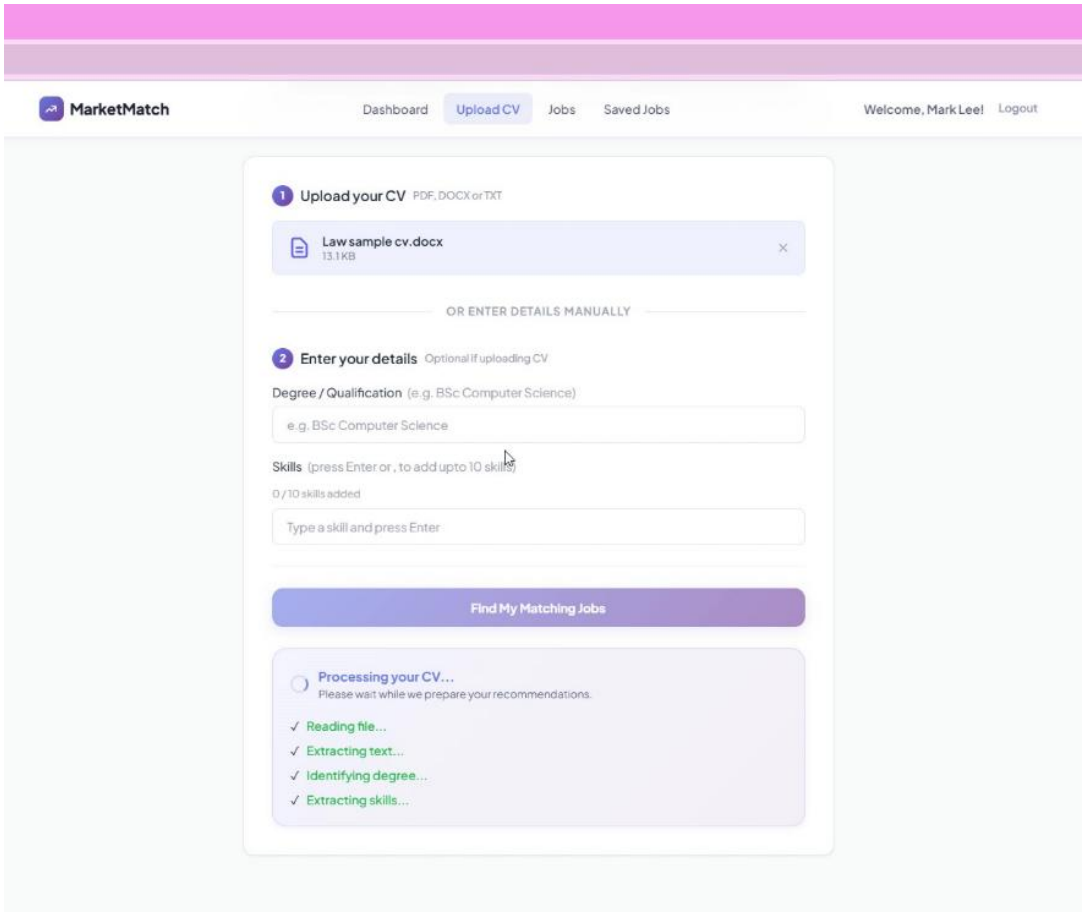


Figure 11: CV Upload Page (upload.html) - Drag-drop and manual entry pathways with processing animation (Source: MarketMatch, 2026)

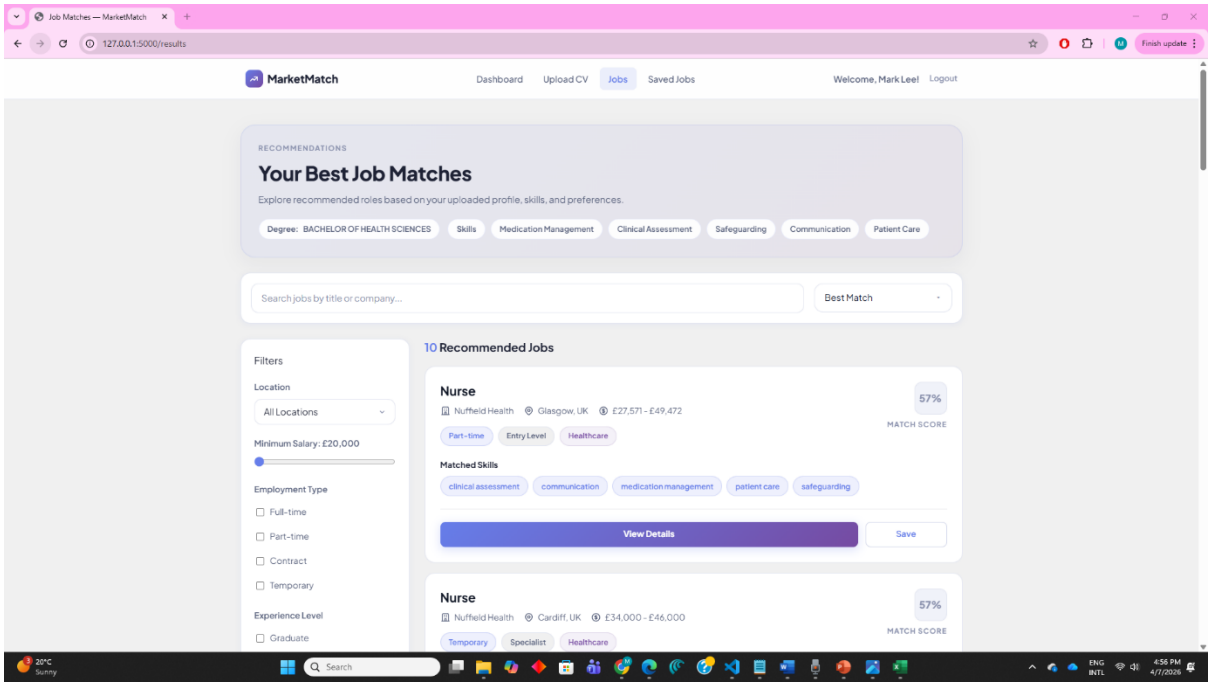


Figure 12: Filtered Job Results - Employment type and salary range filters applied, showing 2 results (Source: MarketMatch, 2026)

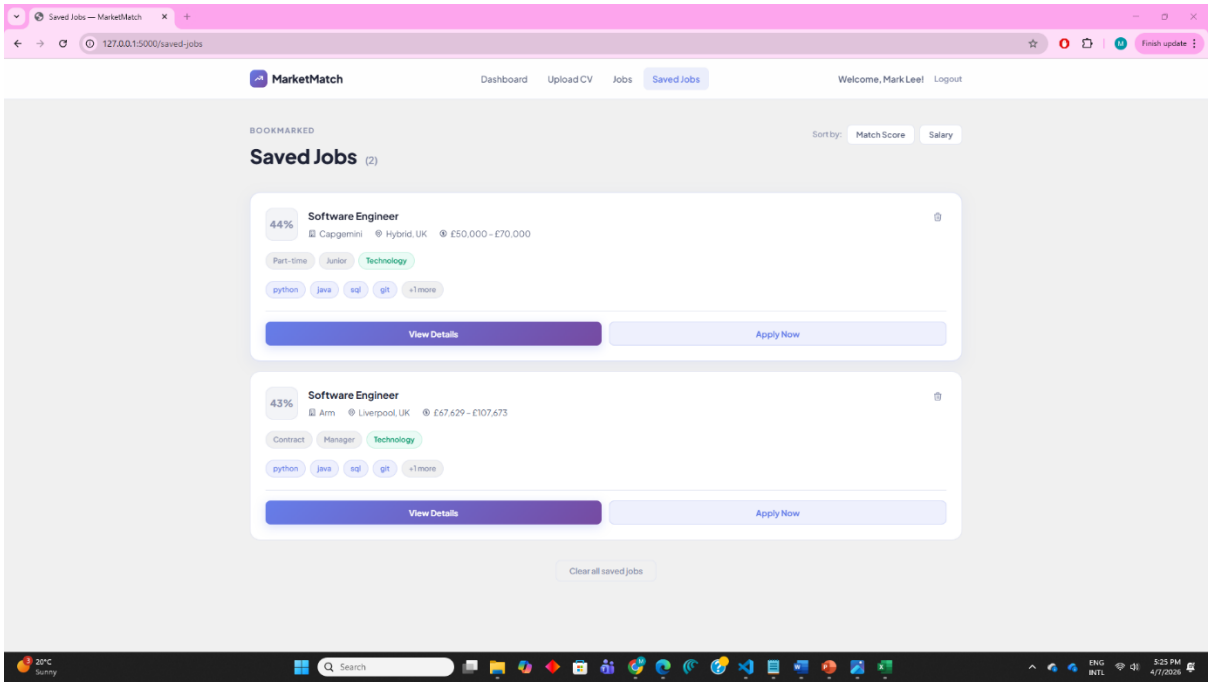


Figure 13: Saved Jobs Page (saved-jobs.html) - Bookmarked roles with sort controls and Apply Now links (Source: MarketMatch, 2026)

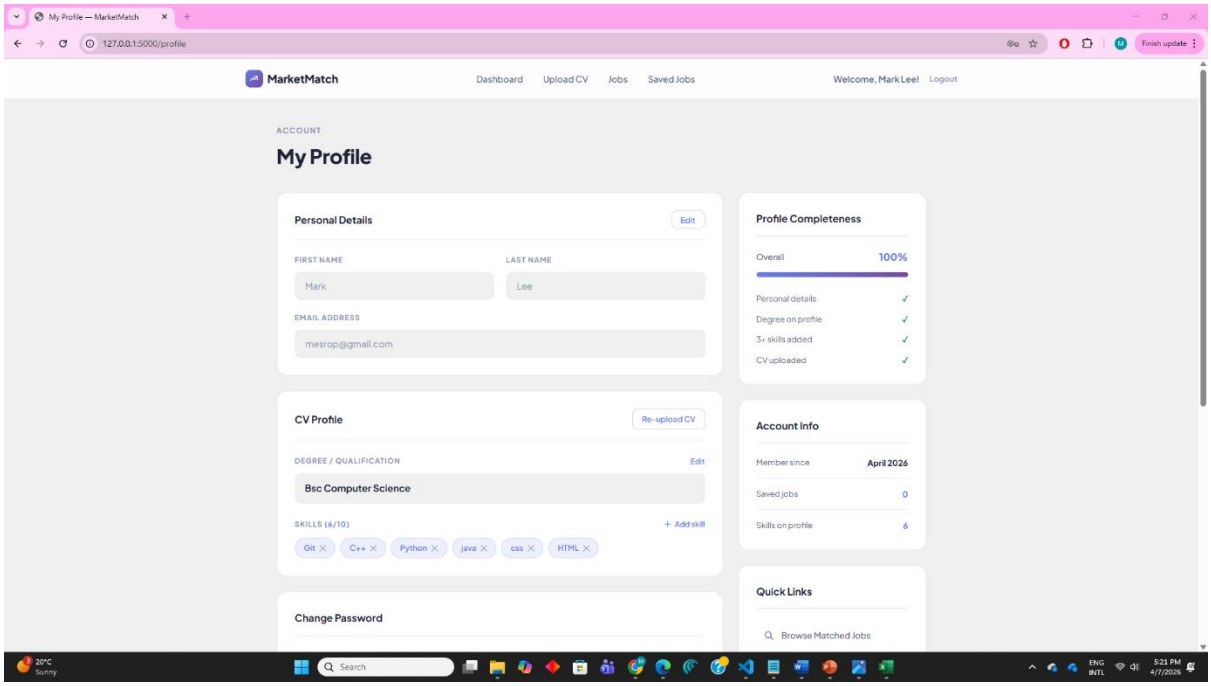
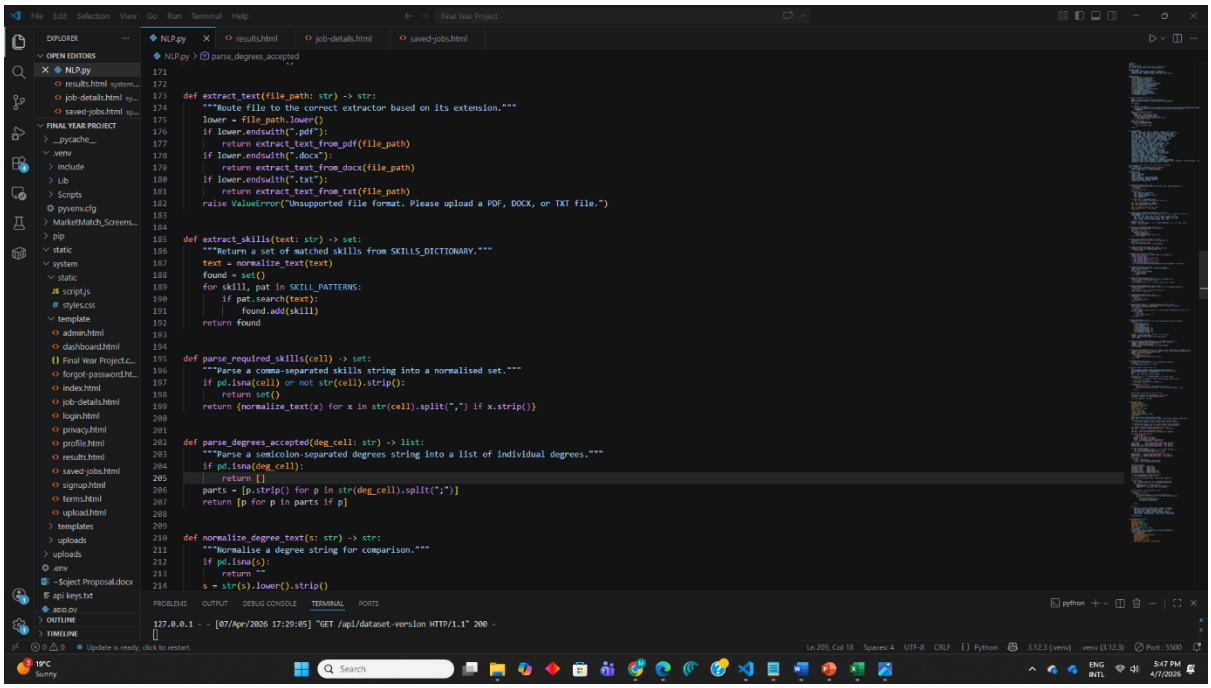


Figure 14: User Profile Page (profile.html) - Degree, skills, account info, profile completeness tracker (Source: MarketMatch, 2026)

One accessibility limitation of the current implementation is the use of a red-amber-green colour gradient on match score badges without a secondary visual encoding. Users with red-green colour vision deficiency (affecting approximately 8% of male users, as noted by the Web Content Accessibility Guidelines 2.1 success criterion 1.4.1) cannot distinguish a 30% score from a 60% score by colour alone. A future release should supplement the colour encoding with a numerical percentage label

displayed inside the badge at all times and consider adding a pattern or icon secondary cue, consistent with the WCAG 2.1 AA standard that the system should aspire to meet.

Appendix C: Algorithm Pseudocode



```
def extract_text(file_path: str) -> str:
    """Route file to the correct extractor based on its extension."""
    lower = file_path.lower()
    if lower.endswith(".pdf"):
        return extract_text_from_pdf(file_path)
    if lower.endswith(".docx"):
        return extract_text_from_docx(file_path)
    if lower.endswith(".txt"):
        return extract_text_from_txt(file_path)
    raise ValueError("Unsupported file format. Please upload a PDF, DOCX, or TXT file.")


def extract_skills(text: str) -> set:
    """Return a set of matched skills from SKILLS_DICTIONARY."""
    text = normalize_text(text)
    found = set()
    for skill, pat in SKILL_PATTERNS:
        if pat.search(text):
            found.add(skill)
    return found


def parse_required_skills(cell) -> set:
    """Parse a comma-separated skills string into a normalised set."""
    if pd.isna(cell) or not str(cell).strip():
        return set()
    return {normalize_text(x) for x in str(cell).split(",") if x.strip()}


def parse_degrees_accepted(deg_cell: str) -> list:
    """Parse a semicolon-separated degrees string into a list of individual degrees."""
    if pd.isna(deg_cell):
        return []
    parts = [p.strip() for p in str(deg_cell).split(";")]
    return [p for p in parts if p]


def normalize_degree_text(s: str) -> str:
    """Normalise a degree string for comparison."""
    if pd.isna(s):
        return ""
    s = str(s).lower().strip()
```

Figure 15: NLP.py – data extraction and parsing (Source: MarketMatch, 2026)

Appendix D: Dataset Sample

Table 12 presents a representative sample of five records from the dataset.csv, illustrating the data structure, field diversity, and salary range distribution that underpin the NLP recommendation system, and the full dataset is included in the project submission archive alongside its version manifest dataset_version.json. The records shown here span Technology, Healthcare, and Legal sectors to demonstrate cross-industry coverage.

job_id	job_title	company_name	location	industry	employme	experienc	posting_date	job_description	required_skills	degree_required	degrees_accepted	min_salar	max_sala	avg_salary
1	Full Stack Developer	Capgemini	Edinburgh, UK	Technology	Temporar	Senior	1/9/2026	Develop end-to-e	javascript, html, css, sql, rest api, git	BSc Computer Science	BSc Computer Scie	75000	95000	85000
2	Software Engineer	Capgemini	Hybrid, UK	Technology	Part-time	Junior	1/22/2026	Design, develop	python, java, sql, git, agile	BSc Computer Science	BSc Computer Scie	50000	70000	60000
3	Software Engineer	BT	Birmingham, UK	Technology	Full-time	Graduate	10/29/2025	Design, develop	python, java, sql, git, agile	BSc Computer Science	BSc Computer Scie	55000	80000	67500
4	Criminal Defense Lawyer	Slaughter and May	Birmingham, UK	Legal	Contract	Manager	11/17/2025	Represent clients	legal research, litigation, negotiation, writing, communication	LLB Law	LLB Law; MA Crimir	80000	150000	115000
5	Data Scientist	Ocado	London, UK	Data & Analytics	Part-time	Executive	11/2/2025	Analyse complex	python, sql, machine learning, statistics, data analysis, research	BSc Data Science	BSc Data Science; I	85000	130000	107500
6	Carpenter	Vistry	Liverpool, UK	Construction	Contract	Junior	12/11/2025	Carry out carpent	problem solving, teamwork, communication, reporting, organisation	BSc Construction Manageme	BSc Construction M	32000	45000	38500
7	UX Designer	BBC	Edinburgh, UK	Technology	Part-time	Manager	1/21/2026	Research user ne	research, presentation, communication, critical thinking	BSc User Experience Design	BSc User Experienc	65000	85000	75000
8	Product Manager	Deliveroo	Cardiff, UK	Technology	Part-time	Principal	12/24/2025	Own product roa	stakeholder management, agile, presentation, decision making, research	BA Business Management	BA Business Mana	90000	140000	115000
9	Mobile Developer	BBC	Oxford, UK	Technology	Part-time	Entry Level	12/21/2025	Build and improv	java, javascript, git, rest api, agile	BSc Computer Science	BSc Computer Scie	45000	65000	55000
10	Head of Department	Pearson	Edinburgh, UK	Education	Temporar	Entry Level	10/13/2025	Lead department	leadership, stakeholder management, presentation, communication, orga	BA Education	BA Education; MSc	35000	47000	40800
11	Criminal Defense Lawyer	Local Chambers	Birmingham, UK	Legal	Full-time	Manager	11/30/2025	Represent clients	legal research, litigation, negotiation, writing, communication	LLB Law	LLB Law; MA Crimir	80000	150000	115000